

AI RESEARCH ROADMAP

A strictly sequential path from newbie to independent AI research



Thiết kế để một newbie có thể mở tài liệu, làm từ trên xuống và luôn biết: học gì → học đến đâu → làm gì → output nào → khi nào được sang bước tiếp.

Nền evidence: learning science, Stanford/MIT/CMU/Berkeley, reproducibility research, top-tier conference standards, expert research practice, AI × Physics reviews.

MỤC LỤC

AI RESEARCH ROADMAP — SEQUENTIAL PLAYBOOK

0	START HERE.....	3
STEP 1	Chạy một hệ ML hoàn chỉnh.....	5
	GATE 1 – Bài kiểm tra kết thúc STEP 1.....	7
STEP 2	Tự xây và làm chủ ML loop.....	8
	GATE 2 – Bài kiểm tra kết thúc STEP 2.....	15
STEP 3	Đọc và reproduce paper.....	16
	GATE 3 – Bài kiểm tra kết thúc STEP 3.....	21
STEP 4	Thử các hướng nghiên cứu.....	22
	GATE 4 – Bài kiểm tra kết thúc STEP 4.....	24
STEP 5	Chọn specialization tạm thời và xây domain base.....	25
	GATE 5 – Bài kiểm tra kết thúc STEP 5.....	28
STEP 6	Original research trong specialization.....	29
	GATE 6 – Bài kiểm tra kết thúc STEP 6.....	34
APPENDIX A	AI × Physics.....	35
APPENDIX B	90-day paper attempt.....	37
APPENDIX C	AI coding assistants.....	38
APPENDIX D	Evidence map & references.....	39

START HERE — Chỉ cần hiểu trang này trước

QUY TẮC SỐ 1

Làm tài liệu theo đúng thứ tự. Hoàn thành 1.1 → 1.2 → ... → Gate 1, rồi mới sang STEP 2. Tương tự cho toàn bộ roadmap.

KHÔNG CÓ “QUAY LẠI STEP CŨ” THEO KẾ HOẠCH

Kỹ năng của STEP trước được mang theo và dùng ở STEP sau. Nếu đang STEP 3 mà phát hiện hỏng chain rule, bạn vá chain rule rồi tiếp tục STEP 3; không quay lại học lại STEP 2.

0.1 Tuyến đường duy nhất

MỐC	BẠN ĐANG LÀM GÌ	KHI XONG
STEP 1	Chạy một hệ ML hoàn chỉnh	Hiểu loop data → model → loss → evaluation.
G1	Giải thích và chạy lại loop	Được sang foundation thật.
STEP 2	Tự xây và làm chủ ML loop	Không còn phụ thuộc tutorial.
G2	Tự giải một problem ML cơ bản	Đủ nền để làm paper reproduction nghiêm túc.
STEP 3	Đọc và reproduce paper	Biết kiểm tra claim của người khác.
STEP 4-5	Thử hướng → chọn specialization tạm thời	Biết mình sẽ đào sâu ở đâu.
STEP 6	Làm original research	Idea → evidence → manuscript → communicate/submit.

0.2 Cấu trúc cố định của mỗi STEP

1. Học kiến thức nền theo thứ tự đánh số.
2. Làm bài tập bắt buộc ngay sau phần kiến thức liên quan.
3. Tạo project/output cuối STEP.
4. Thi Gate bằng năng lực thực tế, không bằng số giờ học hay số course.
5. Pass Gate mới chuyển sang STEP kế tiếp.

PRIORITY RULE —

Bạn vẫn đi theo đúng thứ tự heading. **MUST** = phải đạt và có thể block Gate. **SHOULD** = học/đọc ở đúng vị trí với độ sâu ghi rõ nhưng không cần mastery hay artifact để qua Gate. **LATER / DOMAIN-DEPENDENT** = nhìn thấy để biết nó tồn tại rồi bỏ qua cho tới khi problem thực sự cần.

0.3 Học sâu đến đâu?

LEVEL	NGHĨA	DÙNG KHI
RECOGNIZE	Biết nó là gì và khi nào xuất hiện.	Kiến thức chưa load-bearing.
UNDERSTAND	Giải thích cơ chế, assumptions và intuition.	Concept cần để đọc/ra quyết định.
DERIVE	Tự suy ra/manipulate equation, theo được các bước toán và kiểm tra assumptions.	Math/statistics load-bearing cho method, loss, inference hoặc theory.
IMPLEMENT	Tự code phiên bản canonical.	Cơ chế cốt lõi cần thực sự hiểu.
USE	Chọn, train, debug và evaluate trong problem thật.	Kỹ năng thực hành.
RESEARCH	Biết failure modes, assumptions, cách stress-test/modify.	Sau khi đã vào research area.

STEP

STEP 1 — Chạy một hệ ML hoàn chỉnh

Đừng bắt đầu bằng hàng chục thuật ngữ AI. Hãy tạo một trải nghiệm end-to-end để mọi khái niệm sau này có chỗ bám.

MỤC TIÊU

Thấy tận mắt một bài toán ML đi từ dữ liệu thô đến prediction và metric.

GATE 1

Bạn tự chạy lại được một pipeline nhỏ và giải thích đúng vai trò của từng phần.

1.1 Python + NumPy tối thiểu

Python execution [MUST]Độ sâu: **USE ở mức cơ bản**

- biến, list/dict, vòng lặp, điều kiện, function;
- import module, đọc CSV/array, lỗi exception cơ bản;
- chạy script và notebook.

Phải làm được: Mở một repo/notebook nhỏ, thay đường dẫn/data/parameter và chạy lại mà không làm hỏng pipeline.

Chưa cần: class design sâu, decorators, async, metaprogramming.

NumPy tối thiểu [MUST]Độ sâu: **USE cơ bản**

- array, shape, dtype, slicing;
- matrix/vector operations và broadcasting ở mức trực quan;
- mean/std, random sampling.

Phải làm được: Nhìn shape của X và y rồi giải thích từng chiều.

Chưa cần: tối ưu vectorization nâng cao.

1.2 Học bộ khái niệm cốt lõi để hiểu pipeline

- **sample / feature / label** — một hàng dữ liệu, thông tin đầu vào, target.
- **train / validation / test** — học tham số, chọn quyết định, đánh giá cuối.
- **model / parameter / hyperparameter** — hàm dự đoán, thứ được học, thứ người nghiên cứu chọn.
- **prediction / loss / metric** — output, tín hiệu tối ưu, thước đo mục tiêu.
- **optimizer / learning rate / epoch / batch** — cơ chế update và đơn vị training.
- **overfitting / generalization** — fit train tốt nhưng không chuyển sang dữ liệu mới.

CHƯA CẦN:

Không học toàn bộ AI map, GAN, diffusion, GNN, PINN, neural operator, RL sâu hay AI for quantum ở STEP 1. Tên gọi không giúp nhiều nếu chưa có một pipeline thật để gắn vào.

1.3 Chạy baseline trước

Simple baseline [MUST]Độ sâu: **USE**

- với classification: logistic regression hoặc một baseline rất đơn giản phù hợp dữ liệu;
- với regression: linear/ridge regression hoặc mean predictor làm sanity check;
- chọn một metric phù hợp và ghi rõ metric đó đo gì.

Phải làm được: Có một con số baseline trước khi train neural network.

1.4 Chạy một neural network nhỏ

- Dùng PyTorch ở mức “chạy được”, chưa cần hiểu toàn bộ autograd.
- Model nhỏ: MLP cho tabular/MNIST-like data là đủ.
- Quan sát train loss, validation metric, prediction examples.
- Thay một hyperparameter có chủ ý và dự đoán kết quả trước khi chạy.

1.5 Ghi lại kết quả trong README

- **Problem:** dự đoán cái gì?
- **Data:** input/target là gì?
- **Baseline:** bao nhiêu?
- **NN:** bao nhiêu?
- **Loss/metric:** tại sao dùng?
- Một điều bạn chưa hiểu hoặc một failure bạn thấy.

GATE 1 – Bài kiểm tra kết thúc STEP 1

GATE 1 Không qua Gate bằng việc “đã xem hết khóa học”. Gate chỉ được tính khi bạn tạo ra output quan sát được.

Điều kiện PASS

1. Tự load dữ liệu và xác định X/y.
2. Tự chia train/validation/test bằng cách hợp lý cho dataset đơn giản.
3. Chạy baseline và một NN nhỏ.
4. Giải thích được model, loss, optimizer, metric và overfitting bằng chính experiment của mình.
5. Có README để một người khác hiểu bạn đã làm gì.

Artifact phải giữ lại

- 01_first_ml_system/ – code + README + figure/loss curve.
- Một trang note: “ML loop hoạt động như thế nào?”.

PASS → sang STEP 2 – Tự xây và làm chủ ML loop.

FAIL → vá đúng kỹ năng còn thiếu rồi thi lại Gate 1; không làm lại toàn bộ STEP.

STEP

STEP 2 — Tự xây và làm chủ ML loop

Đây là foundation thật. Sau bước này bạn phải có thể nhận một problem ML chuẩn và tự xây solution mà không cần tutorial cầm tay.

MỤC TIÊU

Biến hiểu biết rời rạc thành năng lực build → debug → evaluate → report.

GATE 2

Cho một dataset mới, bạn tự chọn baseline/model phù hợp, dựng evaluation đáng tin và giải thích toàn bộ pipeline; neural-loop competence được kiểm tra riêng trên một task thích hợp.

2.1 Programming & Research Environment

Python program structure **[MUST]**

Độ sâu: **USE**

- functions/modules/imports; đọc và sửa một multi-file project nhỏ;
- class/object ở mức đủ đọc nn.Module, Dataset và research utilities; không cần OOP architecture sâu;
- filesystem/path, config/arguments và exception/debugger cơ bản.

Command line + environment **[MUST]**

Độ sâu: **USE**

- chạy Python/script/config từ terminal; pwd/cd/ls, paths, environment variables ở mức thực dụng;
- virtual environment + package install; hiểu requirements/pyproject/lockfile ở mức dùng;
- biết notebook là công cụ khám phá, nhưng pipeline chính phải chạy được như script/repo.

Git basics **[MUST]**

Độ sâu: **USE**

- clone/status/add/commit/diff/log; biết quay về commit/config đã tạo một result;

Phải làm được: clone một repo Python nhỏ, tạo environment sạch, chạy từ terminal, đặt breakpoint/debug một lỗi, sửa một module/class nhỏ và commit thay đổi có diff đọc được [1,2].

Chưa cần: shell scripting sâu, SSH/HPC, containers, CI/CD hay distributed systems; học khi project/compute yêu cầu.

2.2 Linear Algebra dùng cho ML

Vectors & matrices **[MUST]**

Độ sâu: **UNDERSTAND + USE**

- vector, matrix, tensor; shape; matrix multiplication;
- dot product, norm, distance, projection;
- basis, rank, linear transformation.

Phải làm được: Viết shape của mọi tensor trong một linear layer và giải thích phép nhân ma trận đó đang làm gì.

Chưa cần: proof trừu tượng sâu nếu không phục vụ research hiện tại.

Eigen / SVD / PCA [SHOULD]

Độ sâu: **UNDERSTAND + USE cơ bản**

- eigenvalue/eigenvector và hướng biến đổi đặc biệt;
- $SVD\ A = U\Sigma V^T$ và low-rank approximation;
- PCA như projection lên directions có variance lớn.

Nên làm được: Dùng PCA cho một dataset và giải thích geometric meaning của các components.

Chưa cần: numerical algorithms để tính SVD.

Gate note: SVD/PCA rất hữu ích và đặc biệt quan trọng trong nhiều hướng scientific computing, nhưng không block G2 nếu bạn đã vững vectors/matrices, gradients, probability và ML loop.

2.3 Calculus + Optimization

Multivariable calculus [MUST]

Độ sâu: **UNDERSTAND + DERIVE**

- partial derivative, gradient;
- chain rule – load-bearing;
- Jacobian; Hessian ở mức nhận diện/intuition.

Phải làm được: Tự derive gradient của linear/logistic regression và backprop cho MLP 2-layer.

Chưa cần: second-order optimization sâu.

Optimization core [MUST]

Độ sâu: **IMPLEMENT + USE**

- objective/loss; local/global intuition;
- gradient descent, SGD, momentum;
- learning rate, regularization; Adam ở mức cơ chế/practical use.

Phải làm được: Tự implement SGD + momentum cho model nhỏ; giải thích vì sao LR quá lớn/nhỏ gây failure.

Chưa cần: duality, constrained optimization sâu trừ khi branch cần.

2.4 Probability + Statistics cho ML

Probability core [MUST]

Độ sâu: **UNDERSTAND + USE**

- random variable; joint/marginal/conditional;
- independence; expectation; variance; covariance;
- Bernoulli, categorical, Gaussian; Bayes rule.

Phải làm được: Giải thích prediction uncertainty và sampling variability bằng ngôn ngữ xác suất.

Likelihood / MLE / MAP [MUST]

Độ sâu: **UNDERSTAND + DERIVE**

- likelihood khác probability statement ở đâu;

- MLE và MAP; prior ở mức intuition;
- liên hệ Gaussian likelihood $\leftrightarrow$ MSE, Bernoulli likelihood $\leftrightarrow$ cross-entropy.

Phải làm được: Derive được MSE hoặc BCE từ một likelihood đơn giản.

Chưa cần: Bayesian inference nâng cao.

Uncertainty of ML evaluation **[MUST]**

Độ sâu: **USE**

- một score đơn lẻ không nói hết độ biến thiên;
- randomness có thể đến từ data sampling, initialization, hyperparameter selection;
- report repeated runs/interval/spread phù hợp với experiment; không dùng “3 seeds” như định luật.

Phải làm được: Chạy một model nhỏ nhiều lần, quan sát distribution của metric và viết kết luận không vượt quá evidence [10].

Chưa cần: formal asymptotic statistics nếu chưa cần.

Statistical estimation for experiments **[MUST]**

Độ sâu: **UNDERSTAND + USE cơ bản**

- Point estimate vs sampling variability; một metric quan sát được là estimate, không phải “giá trị thật” của method.
- Interval estimate / confidence interval ở mức diễn giải; bootstrap như một công cụ thực dụng khi assumptions/analytic formula không thuận tiện.
- Effect size: báo mức độ khác biệt thực tế, không chỉ “có/không significance”.
- Hypothesis testing: UNDERSTAND + dùng khi research design thực sự cần; biết null/alternative, Type I/II error và multiple-comparison risk ở mức cơ bản.

Uncertainty design **[MUST]**

Độ sâu: **UNDERSTAND + USE**

- Trước khi tính error bar, xác định unit of analysis và estimand: bạn đang uncertainty về samples, subjects/groups, trajectories, datasets, training randomness hay toàn pipeline/model-selection process?
- Tách source of variability: data resampling, initialization/training randomness, hyperparameter/model selection, measurement/annotation noise hoặc environment/hardware khi material; các nguồn này trả lời những câu hỏi khác nhau [10].
- Phân biệt SD (spread của observations/runs), SE (uncertainty của estimator) và CI (range cho parameter/estimand dưới assumptions/procedure đã chọn); luôn ghi cách error bar được tính.
- Resampling phải tôn trọng dependence structure: group/time/entity/trajectory-dependent data thì resample ở đúng unit/group, không bootstrap các row như độc lập nếu assumption đó sai.

Phải làm được: trước một comparison, viết 3 dòng: estimand là gì $\rightarrow$ source of variability nào material $\rightarrow$ resampling/interval nào phù hợp và assumptions nào đang dùng [10,14,24,25].

Phải làm được: Với một comparison nhỏ, báo point estimate + uncertainty/interval phù hợp, gọi đúng SD/SE/CI nếu dùng, và giải thích kết luận nào dữ liệu chưa support [24,25].

Chưa cần: proof asymptotic, measure-theoretic probability hay full mathematical-statistics curriculum.

2.5 Data audit & preprocessing – hiểu dữ liệu trước khi chọn model

Raw-data audit [MUST]Độ sâu: **USE + CHECK**

- Mở và nhìn sample thật; xác định đơn vị, range, distribution và ý nghĩa từng feature/label.
- Kiểm tra missing values, duplicates/near-duplicates, label noise, class imbalance, outliers và các nhóm/time/entity dependencies có thể làm observations không độc lập.
- Ghi data provenance/version: dữ liệu đến từ đâu, được thu thập/lọc như thế nào, có thay đổi giữa các run không.

Preprocessing discipline [MUST]Độ sâu: **USE**

- Mọi transformation học từ dữ liệu (normalization statistics, imputation, feature selection...) phải fit trên train rồi mới áp dụng cho validation/test.
- Chọn split theo cấu trúc sinh dữ liệu thật: random row-split chỉ hợp lý khi independence assumption hợp lý; với time/group/entity/trajectory phải split theo structure.

Phải làm được: viết một data-audit note ngắn trước khi train model: data source → quality risks → split unit → preprocessing → leakage risks [11,13].

Chưa cần: data engineering/warehouse/ETL production-scale.

2.6 Classical ML tối thiểu – để có baseline thật**Linear / logistic / softmax regression [MUST]**Độ sâu: **IMPLEMENT + USE**

- objective, prediction, decision boundary;
- regularization cơ bản;
- multiclass softmax.

Phải làm được: Tự viết ít nhất linear hoặc logistic regression bằng NumPy.

Trees / boosting [SHOULD]Độ sâu: **UNDERSTAND + USE**

- decision tree intuition;
- ensemble/boosting như strong tabular baseline khi phù hợp.

Nên làm được: Biết khi nào phải benchmark neural net với một non-neural model mạnh.

kNN [SHOULD]Độ sâu: **RECOGNIZE + USE cơ bản**

- kNN như sanity baseline;

2.7 Generalization & Model Selection – từ fit model sang đánh giá model**Generalization core [MUST]**Độ sâu: **UNDERSTAND + USE**

- Phân biệt training performance, validation performance và expected performance trên data mới; hiểu underfitting/overfitting như failure của fit/generalization.
- Bias-variance intuition và regularization ở mức đủ để chẩn đoán model đơn giản; không cần proof bound ở giai đoạn này.

Model selection [MUST]Độ sâu: **USE**

- Validation set dùng để chọn model/hyperparameter/early stopping; test set dành cho final evaluation và không được “peeking” lặp lại để ra quyết định.
- Cross-validation khi data ít hoặc khi một split đơn quá bất ổn; chọn scheme phù hợp structure của data, không override group/time constraints.
- Khi compare methods, tuning/search budget phải hợp lý và fair; một model được tune 200 trials không nên so trực tiếp với baseline gần như chưa tune.

Phải làm được: Thiết kế model-selection protocol cho một dataset nhỏ và giải thích chính xác lúc nào validation được dùng, lúc nào test được mở [22,25].

Chưa cần: VC dimension, PAC bounds hay learning theory sâu trừ khi branch/theory yêu cầu.

2.8 Neural network từ scratch**MLP + activations [MUST]**Độ sâu: **IMPLEMENT**

- linear layer, nonlinearity, composition;
- forward pass; logits; loss.

Phải làm được: Tự viết MLP bằng NumPy, không autograd.

Backpropagation [MUST]Độ sâu: **DERIVE + IMPLEMENT**

- computational graph;
- local derivative + chain rule;
- gradient accumulation.

Phải làm được: Gradient-check NumPy implementation với PyTorch/autograd hoặc finite differences.

Architectures beyond MLP [SHOULD]Độ sâu: **UNDERSTAND trước, depth theo problem**

- CNN: convolution, locality, weight sharing;
- attention/Transformer: query-key-value, scaled dot-product attention, residual block;
- RNN/LSTM, GNN, VAE, diffusion: chỉ recognize/understand nếu chưa dùng.

Nên làm được: Có thể đọc một diagram CNN/Transformer và theo được tensor flow.

Chưa cần: Master mọi architecture trước research.

2.9 PyTorch – từ framework thành công cụ của mình**PyTorch core [MUST]**Độ sâu: **USE fluently**

- Tensor, dtype/device/shape;
- autograd; nn.Module; state_dict;
- losses, optimizers; Dataset/DataLoader;
- train/eval mode; checkpoint/resume.

Phải làm được: Tự viết training + validation loop từ file trống, không copy tutorial.

2.10 Training & Debugging – phần dễ bị học thiếu nhất

- **Sanity baseline** – luôn có simplest reasonable baseline trước model phức tạp.
- **Overfit a tiny subset** – nếu model không fit được vài samples, pipeline có thể đang sai.
- **Inspect data/predictions** – đừng chỉ nhìn một scalar metric.
- **Learning curves** – train vs validation để phân biệt optimization/generalization.
- **Common failures** – shape mismatch, wrong labels, bad normalization, NaN/Inf, exploding/vanishing gradients, wrong loss, wrong metric, incorrect train/eval mode.
- **Change one major thing at a time** – giữ experiment interpretable.

2.11 Research-code verification – bắt lỗi trước khi metric đánh lừa bạn

Minimal verification **[MUST]**

Độ sâu: **USE + CHECK**

- Assertions cho shape/dtype/range/label assumptions ở các boundary quan trọng của pipeline.
- Tiny deterministic test hoặc one-batch smoke test cho preprocessing, custom loss/metric và data flow quan trọng.
- Dùng invariants/sanity checks khi có thể: probability sums, symmetry/conservation, non-negativity, monotonicity, known toy case...
- Có một command/script smoke-test từ environment sạch để phát hiện dependency/path/config failure trước full run.

Phải làm được: Cố ý làm hỏng ít nhất một component nhỏ và chứng minh assertion/test bắt lỗi trước khi bạn kết luận từ final metric [9,14].

Chưa cần: production CI/CD, distributed test infrastructure hay full software-engineering stack.

EXPERT PRACTICE

Karpathy khuyến nghị xây end-to-end skeleton + dumb baselines, overfit, rồi mới regularize/tune; đây là workflow practitioner, không phải empirical law [13].

2.12 Honest evaluation & reproducibility – bắt đầu từ bây giờ

Data splitting & leakage **[MUST]**

Độ sâu: **USE + CHECK**

- tách train/val/test theo structure thật của data;
- time/group/entity leakage; preprocessing leakage;
- không random-split mù nếu observations phụ thuộc nhau.

Phải làm được: Tự audit pipeline của mình và giải thích vì sao split không làm test information rò vào training [11].

Reproducible experiment **[MUST]**

Độ sâu: **USE**

- Git discipline (clean commit/hash cho result); environment/dependency pinning;
- config/logging; save seed/randomness settings nhưng không coi seed là toàn bộ uncertainty;

- checkpoint, metrics, figures được tạo lại bằng script;
- README đủ để người khác chạy.

Phải làm được: Clone repo vào môi trường sạch và tái tạo được kết quả chính.

Chưa cần: Kubernetes, model serving, production monitoring.

2.13 Research habits bắt đầu sớm

- Đọc paper ngắn/canonical ở mức nhẹ: problem → method → evidence, không đọc frontier paper cực khó để “chứng minh mình đang research”.
- Mỗi experiment có note ngắn: Question → Setup → Result → Failure → Next.
- Technical English: đọc abstract/method trực tiếp; tra thuật ngữ nhưng không dịch toàn bài.
- Tập hỏi người có kinh nghiệm câu hỏi cụ thể kèm code/result, thay vì hỏi “em nên research gì?”.

GATE 2 – Bài kiểm tra kết thúc STEP 2

GATE 2 Không qua Gate bằng việc “đã xem hết khóa học”. Gate chỉ được tính khi bạn tạo ra output quan sát được.

Điều kiện PASS

1. Tự tổ chức và chạy một project Python nhỏ từ terminal: modules/classes cơ bản, virtual environment/package install, Git clone/commit/diff và debugger; không phụ thuộc notebook-only workflow.
2. Inspect raw data, ghi provenance/preprocessing assumptions và thiết kế split phù hợp trước khi model.
3. Nhận một dataset/problem statement chưa từng làm và tự xác định task + metric.
4. Dựng baseline hợp lý và chọn model family có lý do từ data/task/constraints; không ép neural model nếu problem không justify.
5. Trên một task thích hợp, chứng minh neural competence: tự viết training/validation loop PyTorch, checkpoint và evaluation.
6. Phát hiện/giải thích ít nhất một failure thực tế bằng debugging chứ không chỉ đổi model ngẫu nhiên.
7. Có assertions/tests/smoke-check cho load-bearing code path và đã chứng minh ít nhất một lỗi bị bắt trước final metric.
8. Kiểm tra leakage và split assumptions.
9. Đánh giá variability qua repeated experiments phù hợp và không claim mạnh hơn dữ liệu.
10. Chọn model/hyperparameter bằng validation/CV phù hợp, giữ test set cho final evaluation và compare baselines dưới tuning budget hợp lý.
11. Báo point estimate + uncertainty/interval phù hợp; nêu unit of analysis/estimand, source of variability, resampling assumptions và SD/SE/CI đúng nghĩa; viết report khoảng 2–3 trang + README để người khác rerun.

Artifact phải giữ lại

- 02_own_the_loop/ – reproducible repo.
- G2_report.pdf – problem, baselines, model, evaluation, failure analysis, uncertainty.
- Một checklist “what can break this result?”.
- tests_or_checks/ – assertions, smoke test và minimal verification cho load-bearing pipeline.

PASS → sang STEP 3 – Đọc và reproduce paper.

FAIL → vá đúng kỹ năng còn thiếu rồi thi lại Gate 2; không làm lại toàn bộ STEP.

STEP

STEP 3 — Đọc và reproduce paper

Mục tiêu không còn là “hiểu bài giảng”, mà là biến một claim đã xuất bản thành evidence/artifact bạn có thể kiểm tra và tái dựng.

MỤC TIÊU

Học research apprenticeship qua literature, claim-evidence mapping và reproduction/reconstruction.

GATE 3

Bạn reproduce/reconstruct được một claim load-bearing, giải thích discrepancy/assumptions, tái dựng artifact cốt lõi và challenge claim bằng một stress-test phù hợp evidence shape.

3.1 Học anatomy của paper

- **Abstract:** problem + contribution + headline result.
- **Introduction:** motivation, gap, contributions.
- **Related work:** paper đứng ở đâu trong literature.
- **Method / Theory / Evaluation:** object/model/objective/algorithm, assumptions/theorem structure hoặc measurement/protocol – tùy claim.
- **Evidence:** experiments, proofs, evaluation/data analyses hoặc scientific/domain validation – tùy contribution.
- **Results + limitations:** evidence thực sự support được claim nào.

3.2 Đọc paper theo 3 pass

1. **Pass 1** – 5–15 phút: title, abstract, intro, headings, conclusion, figures. Viết 3 câu: problem / idea / result.
2. **Pass 2** – đọc method/theory/evidence; đánh dấu thuật ngữ, assumption, equation hoặc protocol chưa hiểu; vẽ pipeline/argument map.
3. **Pass 3** – chỉ khi paper đáng đầu tư: reconstruct reasoning load-bearing và map claim → assumptions → artifact/evidence; derive/code/proof/audit chi tiết theo claim type.

3.3 Literature search & mapping – tìm đúng paper trước khi reproduce

- Bắt đầu bằng query mô tả problem + method/domain, không bắt đầu bằng tên một architecture đang hot.
- Dùng survey/review/tutorial để dựng bản đồ ban đầu, sau đó quay về primary papers; survey là navigation aid, không thay thế việc đọc nguồn gốc.
- Backward chaining: lần references để tìm foundational/canonical work. Forward chaining: xem papers mới nào extend, critique hoặc fail trên work đó.
- Phân biệt ít nhất 4 loại: canonical/foundational, representative benchmark, recent frontier, failure/critique. Không cần đọc exhaustive literature trước reproduction.
- Lưu bibliography có cấu trúc (Zotero/BibTeX hoặc tương đương) và mỗi paper một paper card: problem / idea / evidence / assumptions / code-data / relevance.

Phải làm được: từ một seed paper, tìm được lineage cơ bản và giải thích vì sao paper bạn chọn là target reproduction phù hợp [8].

SOURCE

Keshav đề xuất three-pass approach như một phương pháp thực dụng để đọc paper hiệu quả; đây là expert/practice guidance, không phải “luật đọc paper” [8].

3.4 Research integrity & provenance – sạch từ lúc bắt đầu

Citation/provenance **[MUST]**

Độ sâu: **USE**

- Ghi rõ paper/version/code commit/dataset source; phân biệt ý tưởng của source với interpretation của mình; cite primary source khi claim phụ thuộc primary result.
- Không fabrication/falsification. Mọi decision-relevant run/attempt phải vào ledger; invalid/debug run có thể exclude khi có technical reason xác định được, nhưng reason phải được ghi và không được invent sau khi nhìn outcome để loại result bất lợi.
- Không cherry-pick seed/checkpoint/hyperparameter hoặc proof/audit attempt sau khi nhìn outcome rồi trình bày như protocol định trước; phân biệt rõ exploratory decisions với confirmatory/final evidence.
- Kiểm tra data/code license, privacy/consent/IRB và authorship/contribution khi applicable; venue-specific rules được kiểm tra lại trước submission [16,26].

Phải làm được: reproduction log cho phép người khác biết dữ liệu/code đến từ đâu, bạn đã thay đổi gì và quyết định nào được đưa ra sau khi nhìn kết quả.

3.5 Chốt paper/claim, reference materials và exact reproduction target

- Ưu tiên target có đủ paper/supplement/data/code hoặc mathematical detail để tái tạo một claim load-bearing trong resources của bạn.
- Xác định paper version (conference/arXiv), supplement, dataset version và code commit/tag nếu có; chốt chính xác table/figure/theorem/lemma/claim bạn sẽ reproduce.
- Nếu code tồn tại, đọc README/issues để biết quirks; nếu code không có/không usable, ghi rõ implementation/proof ambiguities và assumptions bạn buộc phải chọn.

3.6 ESTABLISH REFERENCE – run official code nếu có, reconstruct nếu không

- Nếu official code tồn tại và usable: dựng environment sạch, pin version/commit và chạy config nhỏ trước. [5,32]
- Coi mọi code bên ngoài như untrusted code; nếu cần chạy, dùng secure isolated environment như container/VM/network-isolated instance, và thận trọng với link đi kèm paper [16,31].
- Nếu official code không tồn tại/không usable: reconstruct faithful reference artifact từ paper + supplement – implementation, proof/derivation skeleton, evaluation/data protocol hoặc domain/scientific setup – và ghi mọi ambiguity/assumption.
- Ghi provenance của reference: paper version, code commit nếu có, software/hardware/command khi computational, hoặc theorem dependencies/assumptions khi theoretical.
- Chưa thay đổi method/claim cho tới khi bạn đã thiết lập được reference đủ faithful để biết discrepancy đến từ đâu.

3.7 REPRODUCE – tái tạo một result/claim chính

- Chọn một result/claim rõ ràng, không cố reproduce cả paper: empirical/computational → table/figure/result; theoretical → theorem/lemma/derivation; evaluation/data → benchmark/measurement/audit finding; scientific → solver/domain comparison hoặc validation finding.
- Empirical/computational: đặt tolerance hợp lý dựa trên stochasticity/implementation/hardware. Theoretical: kiểm tra assumptions, proof dependencies, edge cases và counterexamples thay vì giả vờ exact numerical tolerance là universal.
- Nếu lệch: phân loại discrepancy theo claim/evidence shape – data/code/hyperparameter/randomness/undocumented detail; missing assumption/proof gap; measurement/protocol ambiguity; hoặc domain-model/solver mismatch.

3.8 UNDERSTAND – map CLAIM <-> LOAD-BEARING ARTIFACT

- Map từng load-bearing claim sang artifact kiểm chứng nó: computational → equation/description ↔ code/data flow; theoretical → statement ↔ assumptions/lemmas/proof; evaluation/data → construct ↔ measurement/protocol; scientific → domain/physical claim ↔ solver/data/validation evidence.
- Tự vẽ một claim-artifact map: claim nào phụ thuộc artifact nào, input/assumption nào đi vào, output/evidence nào đi ra.
- Giải thích phần nào là essential hypothesis/assumption, phần nào chỉ là engineering/presentation choice, và artifact nào nếu sai sẽ làm claim chính sụp.

3.9 REIMPLEMENT / RECONSTRUCT – load-bearing artifact

- Computational: đóng implementation gốc và viết lại core/minimal faithful version. Theoretical: đóng proof rồi reconstruct proof skeleton/derivation. Evaluation/data: reconstruct measurement/benchmark/audit protocol. Scientific: reconstruct classical/domain baseline, solver interface hoặc validation setup load-bearing.
- Giữ controlled setting/reference cố định để tách misunderstanding khỏi protocol; dùng toy cases/edge cases/counterexamples, benchmark sanity checks hoặc known physical solutions/invariants theo claim type.
- So với reference thích hợp: official code/invariants/toy cases; theorem statement + assumptions + intermediate lemmas; benchmark specification/measurement invariants; hoặc classical solver/known physical checks.

3.10 STRESS-TEST – challenge assumption/regime/claim

1. Chọn một assumption, regime, condition hoặc claim dependency có ý nghĩa.
2. Viết expectation trước khi test: empirical/scientific dự đoán outcome; theoretical dự đoán assumption nào load-bearing hoặc counterexample có thể xuất hiện; evaluation/data dự đoán slice/protocol nào sẽ làm conclusion đổi.
3. Challenge theo evidence shape: empirical → đổi data/noise/scale/hyperparameter/regime; theory → relax/remove assumption, test boundary case hoặc search counterexample; evaluation/data → thay measurement/slice/annotation/protocol; scientific → thay physical parameter/geometry/resolution/boundary/OOD regime.
4. So với control/reference phù hợp và phân tích vì sao claim giữ hoặc vỡ; không chỉ báo một scalar metric hay một failed proof attempt.

3.11 CHALLENGE + ASK – ablate, relax assumption hoặc audit

- Empirical/computational: bỏ/thay component để hỏi nó có thực sự cần thiết không; theory: bỏ/relax assumption hoặc thử alternative lemma/proof route; evaluation/data: audit construct, slice hoặc protocol; scientific: challenge domain assumption/baseline.
- Kiểm tra comparator có đủ mạnh/fair theo claim type: baseline/model; prior theorem/bound; benchmark/measurement; hoặc classical/domain solver.
- Viết ít nhất một câu hỏi/conjecture/audit target nhỏ mà paper chưa trả lời.
- Nếu result âm, vẫn ghi lại; negative result có giá trị để hiệu chỉnh hypothesis.

3.12 Structured failure analysis – tìm pattern thay vì chỉ nhìn global metric

Structured failure analysis **[MUST khi applicable]**

Độ sâu: **USE + RESEARCH**

- Đi từ global metric → class/regime/condition slices → representative failures → recurring pattern → mechanism hypothesis → targeted test.
- ML thông thường: slice theo class, target range, noise level, domain/source, length/scale hoặc condition có ý nghĩa; tránh subgroup mining vô mục đích.
- Scientific ML: slice theo parameter regime, geometry, timestep/resolution, rollout horizon, noise, boundary/initial condition hoặc physical regime.
- Nếu một failure pattern được phát hiện post-hoc, ghi nó là exploratory; muốn nâng thành claim thì kiểm tra lại bằng targeted/held-out/repeated evidence thích hợp.

Phải làm được: tạo một failure table/figure và biến ít nhất một recurring pattern thành hypothesis + targeted test [31].

3.13 Viết reproduction report theo format paper

- Abstract ngắn.
- Original claim.
- Setup và deviations.
- Reproduced result.
- Discrepancies.
- Challenge/stress-test phù hợp claim: ablation, assumption relaxation/counterexample, audit hoặc domain/regime check.
- Question tiếp theo.

3.14 Scientific communication I – trình bày reproduction

- Chuẩn bị talk 5–8 phút: problem → original claim → setup → reproduced result → discrepancy/stress-test → question tiếp theo.
- Một figure/table phải có message rõ; không đọc slide. Kết thúc bằng 1–2 limitation hoặc uncertainty quan trọng nhất.

Phải làm được: trình bày cho peer/mentor/reading group; nếu chưa có audience, record một take rồi tự review clarity và timing.

- Tập Q&A theo claim: “artifact/evidence nào support claim?”, “baseline/prior theorem/measurement/domain comparator có fair không?”, “assumption hoặc alternative explanation nào có thể làm kết luận sai?”.

3.15 Khi nào mở manuscript paper thật?

- Reproduction report tự nó chưa mặc định là novel paper. Nhưng nếu challenge/stress-test tạo một finding không tầm thường, literature check cho thấy câu hỏi chưa được trả lời, và evidence/protocol đủ sạch, hãy mở working manuscript ngay — không đợi STEP 6.
- Manuscript lúc này có thể chỉ có: working title, one-sentence contribution, Related Work skeleton, Method/Theory/Evaluation skeleton và một Evidence/Findings placeholder.
- Nếu chưa có novelty/evidence, giữ artifact ở dạng reproduction report và tiếp tục STEP 4; không ép novelty giả chỉ để “có paper”.

RESEARCH STANDARD

Berkeley liệt kê “critically revisit a published paper” bằng cách reproduce methods/experiments/analysis như một dạng ML research project hợp lệ; JMLR/NeurIPS nhấn mạnh reproducibility là điều kiện để kiểm chứng findings [5,9,14].

GATE 3 – Bài kiểm tra kết thúc STEP 3

GATE 3 Không qua Gate bằng việc “đã xem hết khóa học”. Gate chỉ được tính khi bạn tạo ra output quan sát được.

Điều kiện PASS

1. Chọn được một paper phù hợp năng lực và định nghĩa rõ target reproduction.
2. Thiết lập reference: dùng official artifact nếu available/usable; nếu không, reconstruct faithful implementation/proof/evaluation/domain artifact; reproduce ít nhất một result/claim chính với discrepancy/tolerance/assumptions được giải thích.
3. Map được load-bearing claim sang artifact phù hợp: equation/code, assumptions/lemmas/proof, evaluation protocol/measurement, hoặc domain/validation evidence.
4. Tự reimplement/reconstruct load-bearing artifact: computational core/minimal faithful version; theory proof/derivation; evaluation/data protocol; hoặc scientific/domain baseline/validation setup.
5. Thực hiện ít nhất một stress-test có hypothesis viết trước.
6. Khi result có heterogeneity/failure cases meaningful, thực hiện structured failure analysis và biến một pattern thành targeted test; nếu không applicable, ghi rõ vì sao.
7. Có citation/data/code provenance rõ; không cherry-pick run và ghi lại deviations khỏi protocol.
8. Viết reproduction report nêu discrepancy, provenance/deviations và một question mới.
9. Trình bày reproduction 5–8 phút và trả lời được các câu hỏi cơ bản về claim/evidence/limitations.

Artifact phải giữ lại

- 03_reproduction/<paper_name>/ – code/proof/evaluation/domain artifacts + environment/notes/results as applicable.
- reference_notes.md – official-code status, paper/supplement versions, reconstruction assumptions và ambiguities.
- decision_ledger.md – decision-relevant run/proof/audit attempt, status valid/invalid, exclusion reason, decision impact và compute estimate khi material.
- Reproduction report 3–6 trang.
- Research-question list v1.
- failure_analysis.md/figure – structured slices/regimes + representative failures khi applicable.
- Short reproduction talk: slides/outline + self/peer feedback note.

PASS → sang STEP 4 – Thử các hướng nghiên cứu.

FAIL → vá đúng kỹ năng còn thiếu rồi thì lại Gate 3; không làm lại toàn bộ STEP.

STEP

STEP 4 — Thử các hướng nghiên cứu

Không chọn *specialization* bằng tên gọi hay *trend*. Chọn bằng trải nghiệm: *đọc* → *build* → *reproduce/probe* → *tự đánh giá*.

MỤC TIÊU

Tạo đủ contrast giữa vài candidate branch để biết branch nào đáng đầu tư sâu.

GATE 4

Có một shortlist được kiểm chứng bằng artifacts và một branch đúng đầu để chuyển sang provisional specialization.

4.1 Tạo shortlist – chỉ vài hướng, không phải toàn AI

- Bắt đầu từ mục tiêu thật: dạng problem bạn muốn giải, resources/constraints bạn có (data/compute, theory prerequisites, domain access) và mentor/lab/feedback bạn có thể tiếp cận.
- Tạo một shortlist vài candidate branch đủ để có contrast nhưng không biến exploration thành một curriculum breadth mới; số lượng cụ thể là heuristic, không phải finding khoa học.
- Với AI × Physics, candidate có thể là Scientific ML/PDE, computational imaging/inverse problems, AI for quantum/HEP/astrophysics/materials... tùy comparative advantage.

4.2 SHALLOW SCREEN – loại nhanh bằng literature + feasibility

SCREEN	LÀM GÌ	OUTPUT
A. Landscape	1 review/tutorial tốt + 1 representative paper; map problem/method families.	1 trang field map.
B. Feasibility	Kiểm tra resources/constraints + feedback access + prerequisite gap lớn nhất: data/compute, proof/theory, domain/tooling tùy branch.	Feasibility note.
C. Research signal	Tìm 1 assumption/failure/open question bạn hiểu được ở mức concept.	3–5 dòng signal memo.

4.3 DEEP PROBE – chỉ top 1–2 branch sống sót

PROBE	LÀM GÌ	OUTPUT
A. Papers	1 canonical/representative + 1 recent hoặc failure/critique paper.	2 paper cards.
B. Representative artifact	Artifact rẻ nhưng faithful theo branch: mini implementation/run; proof/derivation reconstruction; benchmark/data audit; hoặc domain/scientific feasibility check.	Repo/notebook/proof/audit note nhỏ.
C. Research signal	Stress-test nhỏ hoặc phân tích assumption/failure; viết câu hỏi bạn thực sự muốn kiểm tra.	Question memo + evidence snippet.

4.4 Chấm branch bằng rubric – không có ranking universal

- **Problem fit** – bạn có muốn giải loại problem này sau khi đã làm thử, không phải trước khi thử?
- **Skill leverage** – toán/CS/physics hiện tại giúp bạn ở đâu?
- **Mentor/feedback** – có người đủ hiểu để phản biện bạn không?
- **Resources / tractability** – với data/compute, proof/theory prerequisites, domain/tooling/feedback access và thời gian hiện có, bạn có một câu hỏi nhỏ nhưng thật và tạo được evidence đáng tin không?
- **Research space** – failure/open problem có rõ không?
- **Literature accessibility** – bạn đọc representative paper mà không cần “dịch toàn bộ field” không?

EVIDENCE BOUNDARY

Không có evidence cho một thứ tự universal kiểu “mentor > interest > compute...”. Hãy coi đây là multi-objective decision. Stanford research rotations cho thấy việc thử nhóm trước khi align advisor là institutional practice, không chứng minh số probe tối ưu [4].

4.5 So sánh bằng evidence, không bằng cảm giác

1. Đặt artifacts của các branch cạnh nhau.
2. Viết 5–10 dòng: điều gì khiến branch này dễ/khó hơn sau khi đã làm thật?
3. Loại branch vì constraint thật (resources, feedback access, prerequisite gap, domain access, mismatch), không vì một paper khó.
4. Chọn 1 branch đứng đầu và 1 backup.

GATE 4 – Bài kiểm tra kết thúc STEP 4

GATE 4 Không qua Gate bằng việc “đã xem hết khóa học”. Gate chỉ được tính khi bạn tạo ra output quan sát được.

Điều kiện PASS

1. Đã shallow-screen nhiều hơn một branch bằng cùng protocol cơ bản.
2. Chỉ top 1–2 branch được deep-probe bằng paper pair + representative artifact phù hợp branch (implementation/run, proof reconstruction, benchmark/data audit hoặc scientific feasibility check) + research signal; không bắt mọi branch trả cùng một chi phí.
3. Có comparison memo dựa trên problem fit, feasibility, mentor/feedback, resources và research space.
4. Có một branch đứng đầu đủ tốt để dành phần lớn research effort cho nó trong một giai đoạn thử nghiệm túc.

Artifact phải giữ lại

- 04_branch_probes/ – shallow notes cho mọi candidate; deep-probe artifacts cho top 1–2 branch.
- branch_comparison.md hoặc PDF 1–2 trang.
- Primary branch + backup branch.

PASS → sang STEP 5 – Chọn specialization tạm thời và xây domain base.

FAIL → vá đung kỹ năng còn thiếu rồi thi lại Gate 4; không làm lại toàn bộ STEP.

STEP

STEP 5 — Chọn specialization tạm thời và xây domain base

“Chọn” ở đây không phải cam kết cả đời. Bạn chỉ quyết định: đây là branch sẽ nhận phần lớn effort cho chu kỳ research tiếp theo.

MỤC TIÊU

Biến một branch đã probe thành domain bạn đủ hiểu để đặt câu hỏi nghiên cứu có cơ sở.

GATE 5

Bạn có domain map, provisional research evidence mode, representative apprenticeship artifact, adversarial-feedback loop và một question/conjecture khả thi để bước vào original research.

5.1 Chốt provisional specialization bằng một decision memo

- Problem class bạn muốn làm.
- Vì sao branch này thắng các probe khác.
- Resources/constraints + mentor/feedback access hiện có: data/compute, theory prerequisites, domain access hoặc tooling – tùy branch.
- Rủi ro chính.
- Ngày review lại quyết định – tránh commit mơ hồ vô thời hạn.

5.2 Xây domain map có cấu trúc

- Central problems / tasks.
- Canonical method families.
- Standard datasets/benchmarks hoặc standard physical/numerical baselines.
- Evaluation protocols.
- Known assumptions.
- Known failure modes.
- Recent frontier và major groups/authors ở mức đủ navigate literature.

5.3 Chọn PROVISIONAL RESEARCH EVIDENCE MODE – evidence shape, không phải venue taxonomy

Không phải mọi AI project có cùng evidence shape. Chọn provisional evidence mode để quyết định loại artifact nào load-bearing. Đây là abstraction của roadmap, KHÔNG phải taxonomy chính thức của NeurIPS/ICML hay bất kỳ venue nào; đến STEP 6.11 mới map project sang contribution type/track hiện hành [30,31].

EMPIRICAL / COMPUTATIONAL —

claim dựa chủ yếu trên experiments/simulations: strong baselines, protocol, uncertainty, ablation/stress-test và reproducibility.

THEORETICAL —

claim dựa chủ yếu trên definitions/assumptions → theorem/conjecture → lemmas/proof/counterexample; empirical sanity checks hữu ích khi relevant nhưng không phải điều kiện universal.

EVALUATION / DATA —

benchmark/dataset/audit/evaluation là contribution chính: construct/measurement validity, coverage, slices/failure modes, leakage, governance/provenance và robustness.

USE-INSPIRED / SCIENTIFIC —

scientific/domain problem là trung tâm: compare ML với strong non-ML/domain baselines, domain validity/V&V và regime/OOD behavior khi relevant.

Phải làm được: viết one-page evidence-mode declaration: provisional mode, load-bearing claim, evidence nào support/refute claim, artifact nào STEP 5–6 cần; hybrid được phép và mode có thể đổi nếu project evidence cho thấy cần.

5.4 Học domain theory theo gap – theo evidence mode, không breadth vô hạn

- Lấy representative paper làm “dependency detector”: khái niệm nào lặp lại và block hiểu biết thì học.
- Math/physics/domain knowledge chỉ đào đến mức claim + evidence mode yêu cầu: method/experiment, proof/assumptions, evaluation construct/protocol hoặc domain/scientific validation.
- Sau mỗi module theory phải có artifact gắn với mode: derivation/proof note, solver/mini experiment, evaluation specification hoặc explanation note.

5.5 Tạo representative apprenticeship artifact theo evidence mode

- Chọn representative work/claim đủ canonical để hiểu standard của branch; artifact phải mirror evidence shape của provisional mode.
- Empirical/computational: reproduce một result/protocol; theoretical: reconstruct theorem/proof/derivation; evaluation/data: reproduce benchmark/measurement/audit protocol; scientific: reproduce solver/physical comparison hoặc validation slice. Không nhất thiết full paper.
- Nếu là AI × Physics/use-inspired, classical/domain baseline và validation phải đủ mạnh; ML-vs-weak-baseline có thể tạo kết luận quá lạc quan [12].

5.6 Thiết lập feedback loop

- Chủ động solicit feedback từ professor/postdoc/senior student/reading group phù hợp; external critique là ưu tiên vì nó phát hiện blind spot mà self-review dễ bỏ qua.
- Gửi representative apprenticeship artifact phù hợp evidence mode + one-page question/conjecture memo; log người/nhóm đã hỏi và objection/feedback nhận được.
- Hỏi câu hẹp: “baseline này có fair không?”, “assumption này có thực sự open không?”, “proof gap/alternative explanation nào nguy hiểm nhất?” thay vì “hướng này có tốt không?”.
- Nếu sau reasonable attempts chưa có external feedback, dùng fallback bắt buộc: structured self-red-team + mock reviewer report + recorded talk/Q&A; external feedback sau đó sẽ supersede self-review khi có.

5.7 Chuyển limitation/gap thành research question hoặc conjecture

1. **Observation/gap:** method, theorem, measurement/benchmark hoặc domain explanation đang fail/uncertain ở đâu?
2. **Mechanism/assumption candidate:** vì sao gap/failure đó có thể tồn tại?

3. **Question/conjecture:** điều gì chính xác chưa biết hoặc cần prove/disprove/audit?
4. **Falsifier:** evidence/counterexample/audit outcome nào sẽ làm hypothesis/conjecture/framing sai?
5. **Minimal feasibility check:** pilot experiment, toy proof/lemma/counterexample search, benchmark slice/audit hoặc domain sanity check rẻ nhất có thể bác/redirect idea.

5.8 Manuscript track – nếu feasibility check đã có signal, viết song song

- Nếu STEP 3/4 đã tạo result có novelty signal, tiếp tục manuscript ngay trong STEP 5: cập nhật novelty sentence, Related Work, protocol và figures khi evidence xuất hiện.
- Không cần đợi “xong specialization” mới viết. Viết sớm giúp lộ logic gap, missing baseline và claim quá mạnh; nhưng manuscript skeleton không phải điều kiện bắt buộc của G5.

5.9 Scientific communication II – domain/problem talk

- Chuẩn bị talk khoảng 10 phút: problem class → domain map → representative methods/results/comparators → failure/open question → proposed question/conjecture + feasibility check.
- Mục tiêu là kiểm tra research framing: người nghe có hiểu “vấn đề nào, vì sao đáng hỏi, evidence nào còn thiếu” trước khi đi vào implementation/proof/audit/domain artifact chi tiết không?

Phải làm được: nhận ít nhất một câu hỏi/objection có substance và ghi nó thành change cho field map, hypothesis hoặc pilot plan [29].

GATE 5 – Bài kiểm tra kết thúc STEP 5

GATE 5 Không qua Gate bằng việc “đã xem hết khóa học”. Gate chỉ được tính khi bạn tạo ra output quan sát được.

Điều kiện PASS

1. Có domain map đủ để đặt một paper mới vào đúng family/benchmark/context.
2. Có representative apprenticeship artifact phù hợp branch/evidence mode: reproduce result/protocol, reconstruct theorem/proof, evaluation/data audit hoặc scientific/domain validation artifact đủ để hiểu field standard.
3. Biết ít nhất một failure mode/assumption có nguồn, không chỉ intuition.
4. Đã chủ động solicit external critique và có adversarial-review artifact; nếu chưa có external response sau reasonable attempts, có structured self-red-team + mock reviewer report + recorded talk làm fallback.
5. Có một research question/hypothesis nhỏ, falsifiable/testable; hoặc với theory, theorem/conjecture precise đủ để prove/disprove và tractable với resources hiện tại.
6. Có feasibility plan + kill/redirect criteria phù hợp evidence mode: pilot experiment, proof bottleneck/lemma/counterexample search, benchmark/data audit hoặc domain validation check.
7. Chốt provisional research evidence mode + load-bearing evidence plan; hiểu rõ đây không phải venue taxonomy và có thể revise nếu project evidence đổi.
8. Trình bày domain/problem talk khoảng 10 phút; ghi external objection nếu có, hoặc substantive mock-review objection nếu phải dùng fallback.

Artifact phải giữ lại

- 05_specialization/field_map.md
- Representative apprenticeship artifact: repo/report, proof reconstruction, evaluation/data audit hoặc scientific validation note.
- One-page research question memo.
- evidence_mode.md – provisional evidence mode, load-bearing claim/evidence, expected artifacts và rationale; ghi rõ hybrid/revision nếu có.
- Feasibility plan + kill/redirect criteria.
- Domain/problem talk outline/slides + feedback note.
- adversarial_review.md – external critique log hoặc structured fallback self-red-team/mock review.

PASS → sang STEP 6 – Original research trong specialization.

FAIL → vá đúng kỹ năng còn thiếu rồi thi lại Gate 5; không làm lại toàn bộ STEP.

STEP

STEP 6 — Original research trong specialization

Từ đây mục tiêu không còn là hoàn thành curriculum. Mục tiêu là biến một câu hỏi thành evidence đủ mạnh để người khác kiểm tra và học được điều gì mới.

MỤC TIÊU

Tự thiết kế và thực thi research project có novelty, soundness, evidence, communication và reproducibility/auditability phù hợp primary research evidence mode.

GATE 6

Bạn đưa một idea từ question/conjecture → methodology/evidence → manuscript → research talk/submission-ready package mà không cần người khác thiết kế toàn bộ process.

ITERATION RULE

STEP 1–5 đi lần đầu theo thứ tự heading. Trong STEP 6, đi 6.1 → 6.11 lần đầu theo thứ tự để dựng full research loop; sau đó được iterate giữa literature ↔ hypothesis ↔ experiment ↔ writing/review. Đó vẫn là đang ở STEP 6, không phải quay lại STEP cũ.

RESEARCH EVIDENCE MODE RULE

Primary evidence mode từ STEP 5 quyết định load-bearing evidence. Empirical/computational → baselines/experiments; theoretical → assumptions/lemmas/proofs/counterexamples; evaluation/data → validity/coverage/audit/benchmark evidence; use-inspired/scientific → domain baselines + validation/V&V. Đây là roadmap abstraction, không phải venue taxonomy; hybrid/revision được phép. Gate 6 kiểm tra soundness theo claim/evidence, không ép một experiment template universal [30,31].

6.1 Problem selection

- **Importance:** nếu giải được thì ai quan tâm và vì sao?
- **Novelty:** question/evidence/perspective mới ở đâu?
- **Tractability:** có thể test meaningful hypothesis với thời gian/compute hiện có không?
- **Falsifiability:** result nào sẽ khiến bạn kết luận hypothesis sai?

6.2 Literature positioning trước khi build lớn

- **Backward citations:** nền tảng mà paper gần nhất dựa vào.
- **Forward citations:** ai đã sửa/criticize/extend work đó.
- **Competing approaches:** ít nhất một family khác.
- Viết novelty sentence thử: “Prior work does X under Y; we test/relax/change Z”. Nếu câu này không đúng vững, chưa nên chạy project lớn.

6.3 Methodology design

ALL MODES:

chốt load-bearing claim, assumptions, evidence standard, controls/comparators và failure criterion trước khi scale.

TARGET SCOPE / EXTERNAL VALIDITY [WHEN APPLICABLE]:

định nghĩa target population/distribution/operating regime và intended use; phân biệt internal/IID generalization với external/OOD transfer, rồi chỉ claim trong phạm vi evidence thực sự cover.

CLAIM TYPE [WHEN APPLICABLE]:

phân biệt descriptive / predictive / causal claim. Association, prediction hoặc feature importance không tự là causal evidence. Nếu claim causal, nêu intervention/exposure + estimand, identification assumptions và study/design logic; nếu chưa có, hạ claim về association/prediction [33].

EMPIRICAL / COMPUTATIONAL:

strong baselines; data split/protocol chống leakage; primary + diagnostic metrics; fair tuning/search budget.

THEORETICAL:

definitions/assumptions rõ; theorem/conjecture; proof plan/lemmas; edge cases và counterexample search; experiment chỉ khi claim cần.

EVALUATION / DATA:

evaluation object, construct/measurement validity, coverage/slices, audit protocol, leakage/governance/provenance và robustness.

USE-INSPIRED / SCIENTIFIC:

strong domain/non-ML baselines, domain constraints, physical/scientific validation, V&V và regime/OOD plan khi relevant.

- Randomness/uncertainty, compute budget và stopping criteria chỉ áp dụng ở mức material cho contribution; theory có thể thay compute bottleneck bằng proof/assumption bottleneck.
- **Compute accounting plan [WHEN MATERIAL]:** ghi hardware/accelerator, memory, wall-clock/accelerator-hours theo major run và ước lượng project-wide compute, gồm preliminary/failed/abandoned runs khi chúng material; không ép compute artifact cho pure theory không dùng computation đáng kể.
- **Domain-specific validation/checks:** conservation/symmetry/physical constraints/OOD, proof sanity/counterexamples, hoặc benchmark/data validity – tùy evidence mode.

6.4 Feasibility first – kill/redirect fast

1. Xây smallest falsifier/feasibility check phù hợp mode: pilot experiment, toy proof/lemma, counterexample search, hoặc mini audit/benchmark slice.
2. Chốt trước signal nào khiến continue / modify / kill-or-redirect.
3. Không scale model/data, proof scope hoặc benchmark construction trước khi feasibility check giải quyết bottleneck chính.
4. Ghi negative result và nguyên nhân; không chỉ giữ run đẹp.

6.5 Build full evidence – theo research evidence mode

EMPIRICAL / COMPUTATIONAL:

lock protocol trước final runs; run strong baselines/proposed method dưới fair budget; ablate/stress-test khi claim phụ thuộc component/regime.

THEORETICAL:

prove main claims từ assumptions đã công bố; kiểm tra proof dependencies, edge cases/counterexamples và relation với known results; empirical validation chỉ khi claim cần.

EVALUATION / DATA:

execute benchmark/data/audit protocol; kiểm tra validity, coverage, leakage, slices/failure modes và robustness của conclusions.

USE-INSPIRED / SCIENTIFIC:

compare ML và strong domain/non-ML approaches; test scientific/physical validity, V&V và relevant regimes/ OOD.

ALL MODES:

report uncertainty/limitations/assumptions phù hợp; không áp statistical, ablation hay compute ritual nếu chúng không bear on the claim.

6.6 Claim-evidence calibration

CORE RULE

Claim strength $\leq$ evidence strength. ICML 2026 reviewer guidance tách soundness, significance, originality và presentation; originality có thể đến từ insight, new task/data/perspective hoặc combination chứ không bắt buộc một architecture hoàn toàn mới [15].

EMPIRICAL / COMPUTATIONAL —

nếu evidence chỉ support một data/regime, không claim universal; kiểm tra baseline fairness, leakage, uncertainty và target distribution phù hợp.

THEORETICAL —

không claim theorem/general result ngoài assumptions đã prove; kiểm hidden assumptions, proof gaps, counterexamples và relation với prior bounds/results.

EVALUATION / DATA —

không generalize ngoài construct/population/coverage được đo; kiểm measurement validity, sampling/ annotation bias, leakage và protocol sensitivity.

USE-INSPIRED / SCIENTIFIC —

claim phải khớp domain validity và intended regime; kiểm strong non-ML/domain comparators, V&V, external/ OOD regimes và alternative physical/domain explanations trước khi mở rộng applicability.

6.7 Reproducibility package

- Artifact package phù hợp mode: code + environment khi computational; proof/derivation supplement khi theoretical; dataset/evaluation card + provenance khi evaluation/data.
- Data acquisition/preprocessing hoặc theorem assumptions/definitions/evaluation protocol — ghi đủ để domain expert audit/reconstruct logic.
- Configs/commands cho computational results; hoặc dependency map cho proofs/lemmas; hoặc scripts/specification regenerate benchmark/evaluation.
- Randomness/tuning/statistical protocol khi stochastic; proof-check/counterexample protocol khi theoretical; sampling/annotation/audit protocol khi data/evaluation.
- **Experiment/decision ledger [WHEN MATERIAL]:** mỗi decision-relevant run/proof/audit attempt có status (valid/invalid/exploratory/final), reason-for-exclusion nếu có, decision impact và approximate compute/cost; invalid technical runs có thể exclude nhưng không outcome-based cherry-pick.

- **Compute record [WHEN MATERIAL]:** hardware, runtime, memory/storage, per-run cost + project-wide estimate đủ để reviewer hiểu resource scale; nêu riêng nếu preliminary/failed/abandoned experiments làm tổng project compute lớn hơn final reported runs [14,16].
- Checkpoints/model details khi relevant; theorem statements/lemmas/assumptions hoặc data schema/versioning khi đó mới là load-bearing artifact.
- Regeneration path cho tables/figures/proofs/benchmark summaries phù hợp contribution.
- Limitations và known failure modes.

CONFERENCE STANDARD

NeurIPS 2026 yêu cầu paper checklist về reproducibility, transparency, ethics và societal impact; checklist không yêu cầu code release trong mọi trường hợp nhưng yêu cầu một reasonable avenue để verify/reproduce contribution [14,16].

6.8 Phát triển manuscript – tiếp tục từ STEP 3–5, không bắt đầu ở đây

Nếu bạn đã mở working manuscript ở STEP 3–5, STEP 6 là lúc nâng nó thành submission-quality document đồng bộ với final evidence; nếu chưa có manuscript vì chưa từng có novelty signal, bắt đầu ở đây.

- Working title + one-sentence contribution.
- **Introduction:** problem → gap → contribution.
- **Related Work:** position, không phải bibliography dump.
- **Method / Theory / Evaluation Protocol:** đủ chính xác để expert reconstruct logic và kiểm tra load-bearing assumptions.
- **Evidence section:** experiments, proofs, evaluation/data analyses hoặc scientific validation – trình bày protocol/assumptions trước storytelling.
- **Results/Theorems/Findings:** mỗi table/figure/theorem/audit trả lời một research question hoặc claim cụ thể.
- **Limitations:** nơi claim không áp dụng.
- **Conclusion:** điều field học được, không phóng đại.

6.9 Internal review → rebuttal mindset

- Tự review theo evidence shape: empirical – baseline/leakage/uncertainty/external validity; theory – hidden assumption/proof gap/counterexample/prior theorem; evaluation/data – construct validity/coverage/measurement bias; scientific – domain baseline/V&V/regime transfer. Sau cùng hỏi: có fatal flaw hoặc unsupported claim nào không?
- Nhờ mentor/peer attack paper, không chỉ sửa wording.
- Lưu “review log”: concern → action → result.

6.10 Scientific communication III – research talk + Q&A

- Chuẩn bị research talk 15–20 phút: problem/context → gap → contribution → methodology → strongest evidence → limitations/failure modes → takeaways.
- Figures phải tự đứng được: mỗi figure trả lời một research question; nói uncertainty/negative evidence thay vì chỉ highlight best result.
- Q&A drill theo evidence shape: novelty; missing baseline/alternative theorem/prior result; leakage/validity; uncertainty; compute fairness khi material; applicability/OOD; fatal alternative explanation/counterexample.

Phải làm được: trình bày cho peer/mentor/reading group và biến ít nhất một objection thành review-log action [29].

6.11 Submission & Peer Review – đưa manuscript vào quy trình khoa học thật

- **Venue/track fit:** đọc Call for Papers, scope, format, anonymity, page limits, supplementary/code/data policy, ethics và AI-use policy của đúng venue/track tại thời điểm submit.
- **Venue mapping:** map roadmap evidence mode sang contribution type/track THỰC TẾ của venue hiện hành; không giả định one-to-one. Nếu venue taxonomy khác roadmap abstraction, venue policy thắng.
- **Submission package:** manuscript, checklist, supplementary, code/data/reproducibility statement và required disclosures; scrub identifying metadata khi double-blind.
- **Pre-submit audit:** novelty positioning, citation coverage, data/code/asset licenses + documentation, consent/IRB/human-subject reporting + appropriate compensation cho participants (crowdsourcing/contract work phải đáp ứng minimum hourly rate của khu vực thực hiện) [14,16], authorship/contribution, dual-submission rule, conflicts of interest, claim-evidence consistency; [WHEN APPLICABLE] broader impacts, safeguards for high-risk release và declaration of material LLM/AI use theo venue policy.
- **AI/review integrity:** verify mọi citation/reference và factual claim do AI hỗ trợ; không fabricated/hallucinated references, prompt injection, hidden reviewer manipulation hoặc embedded instruction nhằm tác động hệ thống review [14,16,26].
- **Reviews:** tách concern thành fatal / major / minor; trả lời bằng evidence và clarification, không bằng defensive rhetoric.
- **Author response / rebuttal:** tuân đúng policy của venue; ưu tiên sửa misunderstanding, cung cấp analysis/clarification được phép và không hứa experiment không thể hoàn thành.
- **Sau decision:** nếu accept, camera-ready + artifact/release; nếu reject, phân loại feedback → sửa research → chọn venue mới thay vì chỉ đổi wording [16,26].

GATE 6 – Bài kiểm tra kết thúc STEP 6

GATE 6 Không qua Gate bằng việc “đã xem hết khóa học”. Gate chỉ được tính khi bạn tạo ra output quan sát được.

Điều kiện PASS

1. Tự chọn/thu hẹp một research question, hypothesis hoặc theorem/conjecture có novelty hợp lý.
2. Tự thiết kế methodology/evidence standard phù hợp primary research evidence mode mà không cần người khác dựng toàn bộ process.
3. Thực hiện feasibility check phù hợp mode và có continue/modify/kill-or-redirect reasoning.
4. Tạo load-bearing evidence đủ support claim: experiments/ablations khi empirical, proofs/counterexamples khi theoretical, validity/audit evidence khi evaluation/data, hoặc domain/V&V evidence khi use-inspired.
5. Có reproducible/auditable package + decision ledger phù hợp contribution và research evidence mode.
6. Viết manuscript mà một domain expert có thể review về soundness/originality/significance.
7. Có compute accounting khi computation material, gồm project-wide estimate và preliminary/failed work khi material; nếu không material, ghi N/A có lý do thay vì tạo artifact hình thức.
8. Trình bày research talk 15–20 phút + Q&A và đưa substantive objection vào review log.
9. Có submission-ready package: venue-policy checklist, reproducibility material, disclosures/ethics checks và response plan cho reviewer concerns.

Artifact phải giữ lại

- Research artifact tree + decision ledger: experiment repo, theory/proof notebook, evaluation/data audit workspace hoặc hybrid; ghi valid/invalid/exploratory/final status và decision impact.
- Reproducibility/auditability package phù hợp primary evidence mode.
- Full manuscript / submission-quality draft.
- Internal review log.
- Research talk slides/outline + Q&A/objection notes.
- Compute accounting summary khi material: major runs + project-wide estimate, gồm preliminary/failed/abandoned work nếu đáng kể; N/A rationale nếu pure theory/non-computational.
- Submission package checklist + venue policy snapshot + rebuttal/response notes.

PASS → sang Independent researcher — từ đây không còn một curriculum tuyến tính universal; bạn tiếp tục deepen domain, research taste, synthesis và leadership qua projects.

FAIL → vá đống kỹ năng còn thiếu rồi thì lại Gate 6; không làm lại toàn bộ STEP.

APPENDIX A

AI × PHYSICS: điều chỉnh roadmap mà không tạo “curriculum thứ hai”

NGUYÊN TẮC

Trước specialization, bạn cần ML core + minimum scientific computing. Domain physics depth được học sau khi problem cho biết cần gì. Đừng học cả quantum/optics/fluid/materials chỉ để “chuẩn bị AI × Physics”.

A.1 Minimum scientific-computing bridge – sau G2

- Floating-point representation; round-off vs truncation error.
- Discretization intuition.
- Numerical differentiation/integration.
- Tự viết một explicit ODE integrator (ví dụ RK4).
- Tự solve một PDE 1D đơn giản bằng finite differences và kiểm tra convergence/sanity.
- FFT: frequency-domain intuition + computational role.
- Dimensional analysis / nondimensionalization.

A.2 Khi vào một branch SciML

- Học governing equations và classical solver đủ để baseline không bị yếu.
- Split theo trajectory/system/time/geometry/parameter regime, không random row-split nếu vi phạm independence.
- Evaluation thêm physical validity: conservation, symmetry, stability, long rollout, OOD regimes – tùy problem.
- Đọc cả promise paper lẫn failure/critique paper; PINNs chẳng hạn có documented optimization/failure modes [18].

A.3 Verification vs Validation – hai câu hỏi khác nhau trong Scientific ML

Verification:

implementation/computation có giải đúng bài toán toán học/computational problem mà bạn đã định nghĩa không?

Validation:

model/computational system đó có đủ đúng với physical system/use case mà scientific claim đang nói tới không?

- Flow tối thiểu khi phù hợp: known/analytic solution → unit/invariant tests → numerical convergence → classical-solver comparison → physical validation → parameter/OOD regimes → scientific claim.
- Không dùng “NN fit benchmark tốt” thay cho verification của numerical pipeline hoặc validation của physical claim. V&V protocol cho predictive SciML đang là active research topic; nguồn [28] đã qua peer review nhưng vẫn nên được dùng như guidance mới nổi, không như settled standard.

A.4 Những thứ hoãn tới khi branch yêu cầu

- finite elements, spectral methods, implicit/stiff solvers, multigrid, adaptive mesh;

- quantum many-body/QFT/DFT/turbulence closure/cosmological inference...;
- PINN/neural-operator internals nếu branch bạn không dùng.

LANDSCAPE

Carleo et al. review ML × physical sciences theo nhiều kiểu giao thoa; Karniadakis et al. review physics-informed ML; neural operators là một family cho mappings giữa function spaces và PDE surrogate problems [17,19,20].

APPENDIX B

90-DAY PAPER ATTEMPT (aggressive, gate-based)

KHÔNG PHẢI GUARANTEE

Đây là chiến dịch attempt cho người có nền toán/physics khá và dành effort lớn. Nếu Gate chưa pass, không tự nhận mình đã sang STEP tiếp chỉ vì lịch đã tới ngày đó.

WINDOW	STEP	TRỌNG TÂM	OUTPUT
Days 1-7	STEP 1	First end-to-end ML system	G1 artifact.
Days 8-32	STEP 2	Foundation + own loop	G2 repo/report.
Days 33-52	STEP 3	One focused reproduction	Reproduction report + question; mở working manuscript nếu có novelty signal.
Days 53-62	STEP 4	Fast probes of a few branches	Comparison memo.
Days 63-68	STEP 5	Provisional specialization + domain map	Question/conjecture memo + feasibility plan; manuscript tiếp tục nếu đã mở.
Days 69-90	STEP 6 attempt	Feasibility check → evidence → submission-quality draft → research talk	Phát triển manuscript thành submission-ready; rehearse talk/Q&A nếu evidence survives.

- Paper attempt trong 90 ngày nên ưu tiên scope hẹp: reproduction + carefully justified extension/stress-test hơn là “invent a new architecture from zero”.

EVIDENCE-MODE NOTE: timeline 90 ngày này tối ưu cho empirical/computational hoặc use-inspired paper. Theory/evaluation-data vẫn dùng Gates, nhưng thay “experiment artifacts” bằng proof/counterexample hoặc benchmark/audit artifacts; evidence mode là roadmap abstraction, không phải venue taxonomy.

- Nếu STEP 3 không tạo ra câu hỏi có signal, dùng STEP 4/5 để đổi problem; đừng ép novelty giả.
- Submission chỉ xảy ra nếu evidence đủ sound; timeline không được dùng để giảm standard.

APPENDIX C

AI CODING ASSISTANTS: dùng để tăng tốc mà không rỗng năng lực

EVIDENCE STATUS

Evidence trực tiếp vẫn còn sớm nhưng không còn gần như trống. Một randomized study về học một programming library mới cho thấy AI assistance có thể làm giảm conceptual understanding, code-reading và debugging khi người học delegation quá nhiều; interaction pattern giữ cognitive engagement cho kết quả học tốt hơn. Dùng đây như learning-safety signal, không như định luật universal [27].

GIAI ĐOẠN	CÓ THỂ DỪNG	BẮT BUỘC TỰ VERIFY / COMPETENCE CHECK
STEP 1	setup, error messages, explanation, boilerplate	Giải thích loop bằng lời của chính bạn; tự dự đoán output/thay đổi trước khi chạy.
STEP 2	plot/data wrangling/boilerplate, explanation, review, implementation acceleration	Knowledge-critical component (backprop/training loop/optimizer): phải vượt competence check không phụ thuộc AI – giải thích, debug code lỗi, viết minimal version hoặc predict behavior.
STEP 3	đọc unfamiliar repo/artifact, locate claim <-> code/proof/protocol/domain evidence, debugging/search hints	Không outsource judgment “paper đúng/sai”, novelty/significance; mọi assistant-generated interpretation phải trace về primary paper + load-bearing code/proof/data/protocol.
STEP 4-6	implementation/proof/audit acceleration + independent review	Research judgment vẫn thuộc bạn; mọi assistant-touched result/artifact phải có verification path, provenance và independent sanity check.

APPENDIX D

Evidence map & references

Cách đọc: [Primary/Peer-reviewed] = bằng chứng hoặc chuẩn chính thức; [Institutional] = cách các chương trình mạnh tổ chức training; [Expert practice] = advice từ researcher, hữu ích nhưng không phải causal evidence; [Synthesis] = quyết định thiết kế roadmap được suy ra từ các nguồn trên.

D.1 Những quyết định nào có evidence mạnh?

DESIGN CHOICE	EVIDENCE	STATUS
Intermediate Python/research environment + math/probability trước ML core	Stanford CS229 yêu cầu Python/NumPy đủ viết chương trình non-trivial; MIT 6.390 yêu cầu intermediate Python/classes + core math [1,2].	Institutional / current.
Research bắt đầu song song coursework	CMU ML PhD active research from first semester [3].	Institutional / explicit.
Explore trước commitment	Stanford first-year research rotations [4].	Institutional; exact probe count = heuristic.
Reproduction là bridge hợp lệ	Berkeley ML project options; JMLR reproducibility [5,9].	Institutional + peer-reviewed.
Leakage phải học sớm	Kapoor & Narayanan, Patterns 2023 [11].	Peer-reviewed systematic evidence.
Uncertainty source > “just seeds”	Bouthillier et al., MLSys 2021: data sampling, initialization và HPO/model-selection variability khác nhau [10].	Peer-reviewed empirical/modeling.
SciML cần strong classical baselines	McGreivy & Hakim, Nat. Mach. Intell. 2024 [12].	Peer-reviewed systematic review, PDE-specific.
Generalization/model selection phải explicit	Stanford CS229 generalization/model-selection notes [22].	Institutional/course-standard.
Research statistics > repeated seeds	CMU ML new curriculum + 36-705; REFORMS reporting framework [23–25].	Institutional + peer-reviewed framework.
Research integrity + submission lifecycle	NeurIPS 2026 handbook; ICML 2026 author instructions [16,26].	Current venue standards.
AI assistance cần competence checks	Randomized AI-coding learning study [27].	Empirical; task-specific, emerging.
SciML tách verification khỏi validation	Predictive SciML V&V recommendations [28].	Peer-reviewed / emerging guidance.
Scientific communication là research competence	CMU ML PhD speaking/writing requirements [29].	Institutional / explicit.

DESIGN CHOICE	EVIDENCE	STATUS
Reproducibility + project compute accounting explicit	JMLR/NeurIPS reproducibility + NeurIPS checklist, including reporting resource scale and relevant total compute [9,14,16].	Peer-reviewed + current venue standard.
Assertions / smoke / intentional fault checks	Engineering synthesis from reliability + reproducibility principles; not a causal training finding.	Synthesis / practical guardrail.
Evidence shape must match claim; roadmap evidence modes $\neq$ venue taxonomy	NeurIPS 2026 reviewer/contribution guidance uses contribution-specific evidence; roadmap compresses this into practical evidence modes [30,31].	Current venue standard + synthesis.
Structured evaluation / failure analysis	NeurIPS 2026 Evaluations & Datasets guidance treats rigorous evaluation, audits, failure modes and limitations as scientific contributions [31].	Current venue standard.

D.2 References

- [1] Stanford CS229 – Machine Learning. Current course/prerequisites and project materials. <https://cs229.stanford.edu/>
- [2] MIT 6.390 – Introduction to Machine Learning (Fall 2026 notes/course overview). <https://introml.mit.edu/fall26>
- [3] Carnegie Mellon University, Machine Learning Department – Ph.D. Curriculum: active research from the first semester, research + courses in parallel. <https://ml.cmu.edu/current-students/phd-curriculum>
- [4] Stanford Computer Science – First-Year Research Rotation Program: up to three groups before permanent advisor alignment. <https://www.cs.stanford.edu/phd-first-year-research-rotation-program>
- [5] UC Berkeley CS 289A – Machine Learning Project: critically revisit/reproduce a published paper is an allowed project form. <https://people.eecs.berkeley.edu/~jrs/189/project.html>
- [6] Sinha, T. & Kapur, M. (2021). When Problem Solving Followed by Instruction Works. *Review of Educational Research*, 91(5), 761–798. doi:10.3102/00346543211019105
- [7] National Academies (2018). *How People Learn II: Learners, Contexts, and Cultures*. National Academies Press.
- [8] Keshav, S. (2007). How to Read a Paper. *ACM SIGCOMM Computer Communication Review*. doi: 10.1145/1273445.1273458
- [9] Pineau, J. et al. (2021). Improving Reproducibility in Machine Learning Research. *JMLR* 22(164):1–20. <https://www.jmlr.org/papers/v22/20-303.html>
- [10] Bouthillier, X. et al. (2021). Accounting for Variance in Machine Learning Benchmarks. *MLSys* 3. <https://proceedings.mlsys.org/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html>
- [11] Kapoor, S. & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* 4, 100804. doi:10.1016/j.patter.2023.100804
- [12] McGreivy, N. & Hakim, A. (2024). Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations. *Nature Machine Intelligence* 6, 1256–1269. doi:10.1038/s42256-024-00897-5
- [13] Karpathy, A. (2019). A Recipe for Training Neural Networks. <https://karpathy.github.io/2019/04/25/recipe/>
- [14] NeurIPS – Paper Checklist Guidelines / Main Track Handbook 2026. <https://neurips.cc/public/guides/PaperChecklist>
- [15] ICML 2026 – Reviewer Instructions: soundness, presentation, significance, originality. <https://icml.cc/Conferences/2026/ReviewerInstructions>
- [16] NeurIPS 2026 – Main Track Handbook, paper checklist for rigor/transparency. <https://neurips.cc/Conferences/2026/MainTrackHandbook>
- [17] Carleo, G. et al. (2019). Machine learning and the physical sciences. *Reviews of Modern Physics* 91, 045002. doi:10.1103/RevModPhys.91.045002
- [18] Krishnapriyan, A. et al. (2021). Characterizing possible failure modes in physics-informed neural networks. *NeurIPS* 34.
- [19] Karniadakis, G.E. et al. (2021). Physics-informed machine learning. *Nature Reviews Physics* 3, 422–440. doi:10.1038/s42254-021-00314-5
- [20] Kovachki, N. et al. (2023). Neural Operator: Learning Maps Between Function Spaces With Applications to PDEs. *JMLR* 24.
- [21] Schulman, J. (2020). An Opinionated Guide to ML Research. <https://joschu.net/blog/opinionated-guide-ml-research.html>

- [22] Stanford CS229 – Main Notes: generalization, bias–variance and model selection/cross-validation. https://cs229.stanford.edu/main_notes.pdf
- [23] Carnegie Mellon University, Machine Learning Department – Ph.D. New Curriculum (effective Fall 2025): Advanced ML + Intermediate Statistics in the first year. <https://ml.cmu.edu/current-students/phd-new-curriculum>
- [24] Carnegie Mellon University 36-705 – Intermediate Statistics: estimation, confidence intervals, hypothesis testing and bootstrap. <https://www.stat.cmu.edu/~siva/teaching/705/>
- [25] REFORMS – reporting standards/framework for machine-learning-based science, emphasizing validity, reproducibility and generalizability. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11092361/>
- [26] ICML 2026 – Author Instructions / author-response and submission policies. <https://icml.cc/Conferences/2026/AuthorInstructions>
- [27] Anthropic Research – AI assistance and coding-skill learning: randomized study on conceptual understanding, code reading and debugging. <https://www.anthropic.com/research/ai-assistance-coding-skills>
- [28] Jakeman, J.D. et al. (2026). Verification and Validation for Trustworthy Scientific Machine Learning, recommendations adapting computational-science V&V to SciML. *Machine Learning: Science and Technology* 7(2), 025055. doi: 10.1088/2632-2153/ae59ec; preprint: <https://arxiv.org/abs/2502.15496>
- [29] Carnegie Mellon University, Machine Learning Department – Ph.D. Requirements: speaking and writing requirements for research communication. <https://ml.cmu.edu/current-students/phd-requirements>
- [30] NeurIPS 2026 – Reviewer Guidelines / contribution-type guidance: theory and other contribution forms are judged by mode-appropriate evidence; empirical validation is not universal for theory. <https://neurips.cc/Conferences/2026/ReviewerGuidelines>
- [31] NeurIPS 2026 – Evaluations & Datasets Reviewer Guidelines: rigorous evaluations, audits, failure modes, datasets/ benchmarks and limitations as scientific contributions. <https://neurips.cc/Conferences/2026/EvaluationsDatasetsReviewerGuidelines>
- [32] ReScience C – journal for computational replication using independent/open implementations; useful evidence that reproduction need not depend on official author code. <https://rescience.github.io/>
- [33] Hernán, M.A., Hsu, J. & Healy, B. (2019). A Second Chance to Get Causal Inference Right: A Classification of Data Science Tasks. *CHANCE* 32(1), 42–49. doi:10.1080/09332480.2019.1579578

D.3 Những phần là heuristic – đừng đọc như định luật

- Strict sequential navigation + Gate order là design/UX constraint để roadmap dễ thi hành; evidence support các skill/dependency lớn, không chứng minh exact serial order 1.1 → 1.2 → ... là causal optimum.
- Số branch nên probe, số paper nên đọc, số ngày cho từng STEP.
- Một ranking cố định giữa interest / mentor / compute / aptitude.
- Một số lượng seed/run universal cho mọi experiment.
- Một mốc tháng universal để “đủ viết paper”.
- Các chi tiết cụ thể của guardrail AI coding assistants; evidence hiện mới nổi và task-specific, nên competence-check principle mạnh hơn một ban-list cố định.
- External-feedback availability và exact evidence-mode artifact shape phụ thuộc field/project; Gate đo năng lực expose work to criticism + evidence quality, không đo network luck.
- Bốn research evidence modes là abstraction của roadmap để chọn evidence shape, không phải taxonomy universal hay taxonomy cố định của venue.
- Exact schema của decision/run ledger và mức compute accounting phải tỷ lệ với contribution; principle là provenance + outcome-independent exclusion, không bureaucracy.

FINAL RULE

Nếu nguồn evidence và một heuristic trong roadmap xung đột, ưu tiên evidence + constraints của research problem. Roadmap là hệ điều hướng, không phải luật tự nhiên.